



13.1. Representation of Graphs

Previous Section:

 [13. Graphs](#)

Contents:

[1. Adjacency List](#)

[2. Adjacency Matrix](#)

[3. Representing Graphs with Python](#)

[3.1. Graph Representation Using an Adjacency List](#)

[3.2. Graph Representation Using an Adjacency Matrix](#)

[Summary](#)

[References](#)

Graphs are powerful data structures capable of representing complex relationships between objects. However, before we can perform operations such as searching, traversing, or finding shortest paths, we must first decide how to store the graph inside a computer.

Unlike arrays, linked lists, stacks, or queues, graphs are not stored directly in memory as a single structure. Instead, they are represented using other data structures.

The two most common graph representations are: Adjacency List and Adjacency Matrix

Both representations store the same graph but differ significantly in memory usage, efficiency, and implementation.

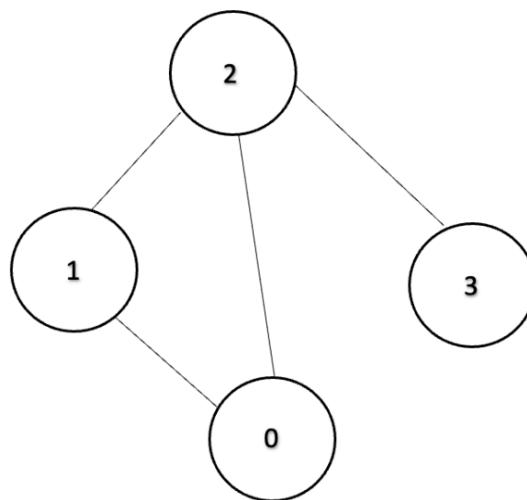
In this section, we will examine both approaches and learn how to implement them in Python.

1. Adjacency List

The **Adjacency List** represents a graph using an array (or dictionary) of linked lists.

Each vertex corresponds to one index in the array, and the linked list attached to that index stores all vertices adjacent to it.

Consider the following graph:



Vertices: 0, 1, 2, 3

Edges: (0, 1); (0, 2); (1, 2); (2, 3)

Building the Adjacency List

First, create an array where each index represents a vertex.

```
0  
1  
2  
3
```

Then connect each vertex to all of its neighboring vertices.

- Undirected Graph: Because an undirected edge can be traversed in both directions: $0 \leftrightarrow 1$, we must store: $0 \rightarrow 1; 1 \rightarrow 0$ inside the adjacency list.

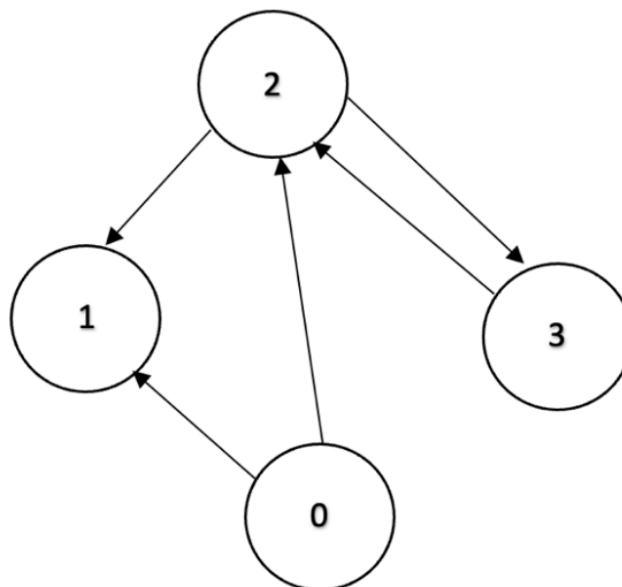
```
Vertex 0 → 1 → 2 → None
Vertex 1 → 0 → 2 → None
Vertex 2 → 0 → 1 → 3 → None
Vertex 3 → 2 → None
```

Or more compactly:

```
0 → 1 → 2 → None
1 → 0 → 2 → None
2 → 0 → 1 → 3 → None
3 → 2 → None
```

- Directed Graph: For directed graphs, edges have directions.

Consider: $0 \rightarrow 1; 0 \rightarrow 2; 2 \rightarrow 1; 2 \rightarrow 3; 3 \rightarrow 2$



- **Adjacency List Representation**

```
0 → 1 → 2 → None
1 → None
2 → 1 → 3 → None
3 → 2 → None
```

Notice: $0 \rightarrow 1$ does not imply: $1 \rightarrow 0$ because directions matter.

How Adjacency Lists Are Stored

Conceptually:

Array		
Index		Linked List
0	→	1 → 2
1	→	0 → 2
2	→	0 → 1 → 3
3	→	2

Modern implementations usually replace linked lists with Python lists:

```
adj = {0:[1,2], 1:[0,2], 2:[0,1,3], 3:[2]}
```

Advantages of Adjacency Lists

- **Memory Efficient:** Only existing edges are stored. For example, 1000 vertices with only 20 edges. The adjacency list stores only those 20 edges.
- **Fast Neighbor Traversal:** Finding all adjacent vertices is straightforward.
 - Vertex 2: $2 \rightarrow 0 \rightarrow 1 \rightarrow 3$
 - Neighbors can be visited immediately.
- **Ideal for Sparse Graphs:** Sparse Graph is a graph with many vertices but few edges. Example being social Network with 100,000 users and 50,000 friendships. Adjacency lists save substantial memory.

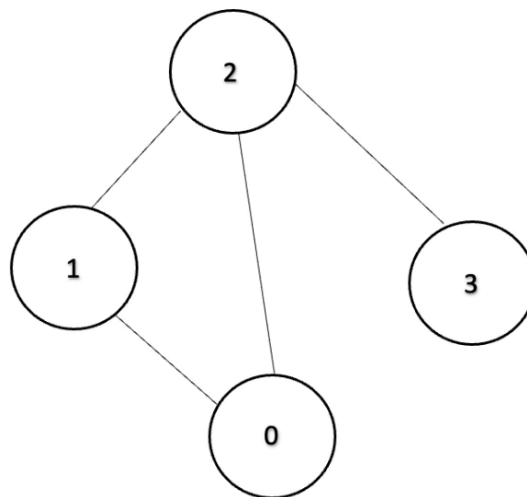
Disadvantages of Adjacency Lists

- **Edge Checking is Slower:** To determine whether $(2, 3)$ exists, we must search through Vertex 2's list $2 \rightarrow 0 \rightarrow 1 \rightarrow 3$
- **More Complex Implementation:** Compared to matrices, adjacency lists require additional structures such as linked lists, arrays, or dictionaries.

2. Adjacency Matrix

The **Adjacency Matrix** stores graph data inside a two-dimensional matrix. Rows and columns represent vertices. Each entry indicates whether an edge exists between two vertices.

Consider the same graph:



- Vertex Labels

```
0 1 2 3
```

- Matrix Layout

```

    0 1 2 3
0
1
2
```

3

Rows indicate source vertices. Columns indicate destination vertices.

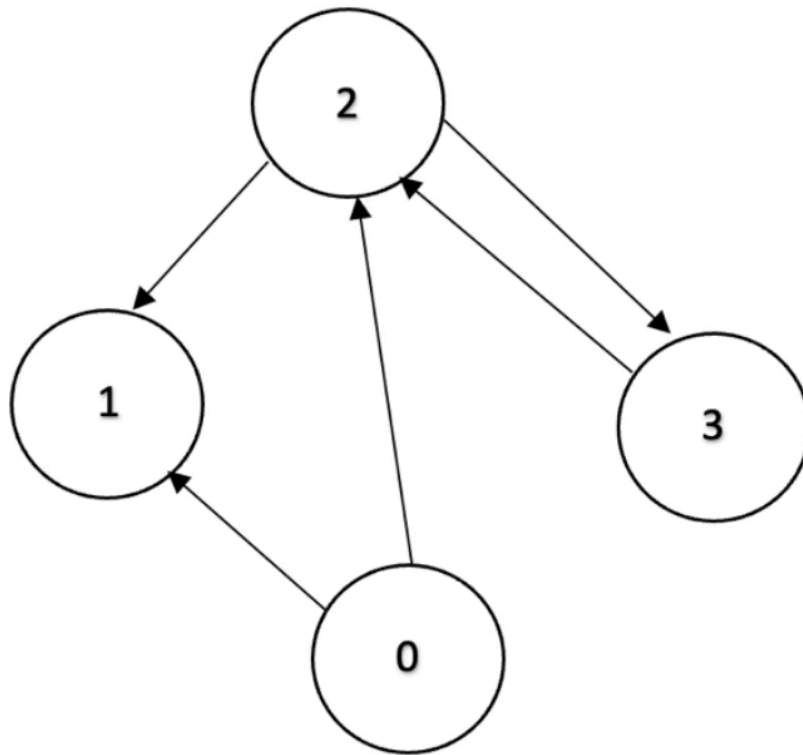
- **Undirected Graph Matrix** - For an undirected graph:

	0	1	2	3
0	0	1	1	0
1	1	0	1	0
2	1	1	0	1
3	0	0	1	0

Why is it Symmetrical? Observe: `Matrix[0][1] = 1` and `Matrix[1][0] = 1`

Since: $0 \leftrightarrow 1$, both entries must be identical. Therefore: An undirected graph representation is always a symmetrical matrix

- **Directed Graph Matrix** - For the same graph but with directions:



$0 \rightarrow 1; 0 \rightarrow 2; 2 \rightarrow 1; 2 \rightarrow 3; 3 \rightarrow 2;$

The matrix becomes:

	0	1	2	3
0	0	1	1	0
1	0	0	0	0
2	0	1	0	1
3	0	0	1	0

Notice that `Matrix[0][1] = 1` but `Matrix[1][0] = 0`

Therefore: For a directed graph representation, the matrix is not necessarily symmetrical

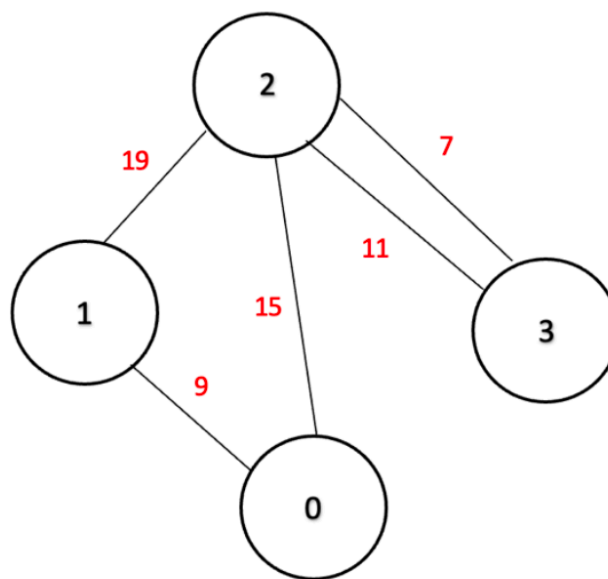
- **Diagonal Entries** - Observe in both cases

- `Matrix[0][0]`
`Matrix[1][1]`
`Matrix[2][2]`
`Matrix[3][3]`

All are: 0 because there are no self-loops. In other words, these paths $0 \rightarrow 0$; $1 \rightarrow 1$; $2 \rightarrow 2$; $3 \rightarrow 3$ do not exist.

```
      0 1 2 3
0    [0] 1 1 0
1    1[0]1 0
2    1 1[0]1
3    0 0 1[0]
```

- **Weighted Graph Matrix:** When edges have weights, the matrix stores weights instead of 1.



Adjacency Matrix

```
      0 1 2 3
0    0 9 15 0
1    9 0 19 0
2    15 19 0 11
3    0 0 11 0
```



```
Matrix[0][1] = 9 -> Weight between Vertex 0 and 1 = 9  
Similarly: Matrix[2][3] = 11 means: Weight(2,3) = 11
```

Advantages of Adjacency Matrices

- **Fast Edge Lookup:** Checking whether an edge like (A, B) exists requires only: `matrix[A][B]` which takes constant time: $O(1)$
- **Simple Representation:** The graph is stored in a single matrix (Rows = Sources; Columns = Destinations). Easy to visualize and implement.
- **Suitable for Dense Graphs:** Dense Graph contains many vertices and many edges. Since most matrix entries are used, memory waste is minimal.

Disadvantages of Adjacency Matrices

- High Memory Usage: A graph with V vertices requires: $V \times V$ storage.
 - Example: 1000 vertices $\rightarrow 1000 \times 1000 = 1,000,000$ entries even if only a few edges exist.
- Poor for Sparse Graphs
 - Example: 1000 vertices and 20 edges $\rightarrow$ Most entries remain: 0 which wastes memory.

Comparison of Graph Representations

Feature	Adjacency List	Adjacency Matrix
Storage	Vertices + Edges	$V \times V$ Matrix
Memory Usage	$O(V + E)$	$O(V^2)$
Edge Lookup	$O(\text{degree})$	$O(1)$
Traversing Neighbors	Fast	Slower
Sparse Graphs	Excellent	Poor
Dense Graphs	Good	Excellent
Implementation	Moderate	Simple

3. Representing Graphs with Python

Now that we understand the theory behind graph representations, let us implement them in Python using a **Graph class**.

Encapsulating graph operations inside a class allows us to create reusable graph objects and perform common operations such as adding edges, displaying the graph, and checking connectivity.

3.1. Graph Representation Using an Adjacency List

An adjacency list can be implemented using a dictionary where:

- Each key represents a vertex.
- Each value is a list containing all adjacent vertices.

```
class Graph:
    def __init__(self):
        self.adj_list = {}

    def add_vertex(self, vertex):
        if vertex not in self.adj_list:
            self.adj_list[vertex] = []

    def add_edge(self, source, destination):
        self.add_vertex(source)
        self.add_vertex(destination)
        # Undirected Graph
        self.adj_list[source].append(destination)
        self.adj_list[destination].append(source)

    def display(self):
        for vertex in self.adj_list:
            print(f"Vertex {vertex}: ", end="")
            for neighbor in self.adj_list[vertex]:
                print(neighbor, end=" -> ")

            print("None")
```

```

g = Graph()

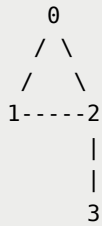
g.add_edge(0, 1)
g.add_edge(0, 2)
g.add_edge(1, 2)
g.add_edge(2, 3)

g.display()

```

Vertex 0: 1 → 2 → None
 Vertex 1: 0 → 2 → None
 Vertex 2: 0 → 1 → 3 → None
 Vertex 3: 2 → None

Visualization



Adjacency List:

```

0 → 1 → 2 → None
1 → 0 → 2 → None
2 → 0 → 1 → 3 → None
3 → 2 → None

```

Checking Whether an Edge Exists

A useful operation is determining whether two vertices are connected.

```

class Graph:

    ...

    def has_edge(self, source, destination):

```

```
if source not in self.adj_list:
    return False

return destination in self.adj_list[source]
```

```
print(g.has_edge(0, 2))
print(g.has_edge(1, 3))
```

True

False

Directed Graph Version

For directed graphs, the edge is only added in one direction.

```
def add_edge(self, source, destination):

    self.add_vertex(source)
    self.add_vertex(destination)

    self.adj_list[source].append(destination)
```

Example:

```
0 → 1
0 → 2
2 → 1
2 → 3
```

Adjacency List:

```
0 → 1 → 2 → None
1 → None
2 → 1 → 3 → None
3 → None
```

3.2. Graph Representation Using an Adjacency Matrix

A graph can also be represented using a two-dimensional matrix. Each entry: `Matrix[i][j]` indicates whether an edge exists between vertex i and vertex j .

```
import numpy as np

class Graph:

    def __init__(self, vertices):
        self.vertices = vertices
        self.matrix = np.zeros((vertices, vertices), dtype=
int)

    def add_edge(self, source, destination):
        # Undirected Graph
        self.matrix[source][destination] = 1
        self.matrix[destination][source] = 1

    def display(self):
        print(self.matrix)
```

```
g = Graph(4)

g.add_edge(0, 1)
g.add_edge(0, 2)
g.add_edge(1, 2)
g.add_edge(2, 3)

g.display()
```

```
[[0 1 1 0]
 [1 0 1 0]
 [1 1 0 1]
 [0 0 1 0]]
```

Visualization

```
0 1 2 3
```

```
0    0 1 1 0
1    1 0 1 0
2    1 1 0 1
3    0 0 1 0
```

Checking Whether an Edge Exists

For adjacency matrices, edge lookup is extremely fast.

```
class Graph:

    ...

    def has_edge(self, source, destination):
        return self.matrix[source][destination] == 1
```

```
print(g.has_edge(0, 2))
print(g.has_edge(1, 3))
```

True

False

Inserting an Edge

Adding a new edge simply requires updating the matrix entry.

```
g.add_edge(1, 3)
```

Result:

```
      0 1 2 3
0    0 1 1 0
1    1 0 1 1
2    1 1 0 1
3    0 1 1 0
```

This object-oriented implementation will also be useful in the next section when implementing graph traversal algorithms such as **Depth-First Search (DFS)** and **Breadth-First Search (BFS)**.

Summary

Graphs cannot be stored directly in memory and therefore require specialized representations. The two most common approaches are the **Adjacency List** and the **Adjacency Matrix**.

The adjacency list stores only existing connections and is highly memory-efficient for sparse graphs, making it suitable for large networks where relatively few edges exist. The adjacency matrix stores graph information inside a two-dimensional matrix, providing extremely fast edge lookups at the cost of higher memory consumption.

Both representations have their strengths and weaknesses, and the choice between them depends on the structure of the graph and the operations that will be performed most frequently.

By combining graphs with other data structures such as arrays, linked lists, dictionaries, and matrices, complex graph operations can be implemented efficiently. These representations form the foundation for graph algorithms such as traversal, shortest-path computation, network analysis, and path-finding, which will be explored in the following sections.

We will learn how to systematically visit vertices in a graph using **Depth-First Search (DFS)** and **Breadth-First Search (BFS)**.

References

https://drive.google.com/file/d/1R4R6eGz_XZEva12XTJEv3oIhZt1aet4x/view?usp=drive_link

<https://www.geeksforgeeks.org/dsa/comparison-between-adjacency-list-and-adjacency-matrix-representation-of-graph/>

<https://www.geeksforgeeks.org/dsa/graph-and-its-representations/>

Next Section:

13.2. Graphs Traversal